

Ch10_Django's ORM vs SQL

Database management:

- ORM (Object Relational Mapper)
 - Higher level of abstraction; More efficient ; Generate slower;
 - SQL
 - Higher hands-on management ; Complex query;
-

Advantages of Django ORM

- A huge improvement in development speed, code maintenance, security and ease of development.
 - Much easier and less error-prone 容易出错 to modify the query
 - By using ORM exclusively you are guaranteed to be safe from SQL injections.
 - It supports seamless schema generation and migration, and queries with them, data validation and integrity, switching between various databases without rewriting code, etc.
-

Raw SQL queries

- You can fall back to writing raw SQL.
- Django gives you **two ways** of **performing** raw **SQL** queries:
 1. Use [Manager.raw\(\)](#) to perform raw queries and return model instances, or
 2. Avoid the model layer entirely and [execute custom SQL directly](#).
- A model manager's [raw\(\)](#) method returns results are structured as a [RawQuerySet](#) class instance, which is very similar to the [QuerySet](#) class instances produced by Django model queries.

Django 提供了两种执行原生 SQL 的方式:

1. 使用 `Manager.raw()` 方法。-- 通过调用模型管理器的`raw()`方法, 执行原生SQL查询并返回模型实例。
2. 绕过模型层, 直接执行自定义的 SQL 语句

▼ raw()

Python

```
1 queryset = MyModel.objects.raw('SELECT * FROM my_table WHERE id=%s', [1])
2 for obj in queryset:
3     print(obj.name)
```

`raw()` 返回的结果是 `RawQuerySet` 类的实例，其行为与 Django 的 `QuerySet` 类似，但直接映射原生 SQL 的输出。

使用 `raw()` 方法

(1) Examples of Django model manager `raw()` method

- By default, Django figures out a database table name by joining the model's "app label" – name used in `manage.py` startapp – to the model's class name, with an underscore between them.
- Performing raw SQL queries in interactive shell
(django)05:34~/django_projects/projectName \$ `python manage.py shell`
>>> `from myapp.models import Person`
>>> `for p in Person.objects.raw('SELECT * FROM myapp_person'): print(p)`
... <Press Enter>
It's exactly the same as running `Person.objects.all()`
- Raw SQL query with aggregate function added as extra model field
>>> `items_with_inventory = Item.objects.raw("SELECT sum(price*stock) as assets from items_item");`
- `raw()` supports indexing, so if you need only the first result you can write:
>>> `first_person = Person.objects.raw('SELECT * FROM myapp_person')[0]`
table

(1) Passing parameters into `raw()`

- We have executed only static queries so far.
- To create SQL queries dynamically, we need to pass user-supplied data into our queries.
- You can use the `params` argument: `[ ]` to pass parameters into `raw()`:
`lname = 'Doe'`
`Person.objects.raw('SELECT * FROM myapp_person WHERE last_name = %s', [lname])`

`user = Users.objects.raw('SELECT * FROM main_users WHERE mobile = %s OR email = %s', [mobile, email])`
- You cannot quote the placeholder `(%s)`, otherwise, it does not work.

=> 通过 `params` 参数将用户提供的数据动态传递给 `raw()` 方法，以执行 动态的 SELECT 查询。

- `[lname]` 是参数列表，表示占位符 `%s` 的具体值。
- 这里的 `'%s'` 是一个占位符，用来表示 SQL 查询的参数。它并不是直接拼接字符串，而是通过安全的参数传递机制避免 SQL 注入。

通过 `connection.cursor()` 直接执行 SQL 查询

(2) Executing custom SQL directly

- You can access the database directly, routing around the model layer. That is, without the need to use `Person.objects.raw('Select...')`
- The object `django.db.connection` represents the default database connection.
- To use the database connection, call `connection.cursor()` to get a cursor object.
- A cursor is an object which helps to execute the query and fetch the records from the database.
- Then, call `cursor.execute(sql, [params])` to execute the SQL and `cursor.fetchone()` or `cursor.fetchall()` to return the resulting rows.

1. 使用 `django.db.connection` :

Django 提供了一个默认的数据库连接对象 `connection`，您可以通过它执行自定义 SQL 查询。

该对象允许通过 `connection.cursor()` 方法获取游标（cursor），游标用于执行 SQL 和获取查询结果。

2. 游标的主要方法：

- `cursor.execute(sql, [params])`：用于执行 SQL 查询。例如，您可以执行插入、更新或选择语句。
- `cursor.fetchone()` 或 `cursor.fetchall()`：用于返回 SQL 查询的结果：
 - `fetchone()` 返回单行数据。
 - `fetchall()` 返回所有查询结果。

```

1  from django.db import connection
2
3  with connection.cursor() as cursor:
4      cursor.execute("SELECT * FROM my_table WHERE id=%s", [1]) # 执行查询
5      rows = cursor.fetchall() # 获取所有查询结果
6      for row in rows:
7          print(row) # 处理查询结果

```

该方法绕过了 Django 的 ORM 提供的抽象层，在某些场合提高了灵活性

cursor.fetchone() or cursor.fetchall()

- **fetchone():** Fetch the next row of a query result set, returning a single tuple, or None when no more data is available


```

>>> cursor.execute("SELECT * FROM myapp_test WHERE id = %s", [3])
>>> cursor.fetchone()
(3, 42, 'bar')

```
- **fetchall():** Fetch all (remaining) rows of a query result, returning them as a list of tuples. An empty list is returned if there is no more record to fetch.


```

>>> cursor.execute("SELECT * FROM myapp_test;")
>>> cursor.fetchall()
[(1, 100, "abc'def"), (2, None, 'dada'), (3, 42, 'bar')]

```

raw() 方法

返回一个 `RawQuerySet`，与模型实例绑定。可以像处理 `QuerySet` 一样迭代结果

```

1  queryset = MyModel.objects.raw('SELECT * FROM my_table WHERE id=%s', [1])
2  for obj in queryset:
3      print(obj.name) # 访问模型字段

```

- 不支持复杂的非模型字段查询。
- 只能用于读取操作（SELECT 查询），不适用于写入（INSERT、UPDATE）或删除操作。

connection.cursor()

- 提供完全灵活的原生 SQL 操作能力。
- 结果类型：
 - 返回原始查询结果（通常是元组列表）。

```
1 from django.db import connection
2 with connection.cursor() as cursor:
3     cursor.execute('SELECT name FROM my_table WHERE id=%s', [1])
4     result = cursor.fetchall()
5 for row in result:
6     print(row[0]) # 手动解析结果
```

- 可执行任何类型的 SQL（包括 SELECT、INSERT、UPDATE、DELETE）。
- 适合操作复杂的数据库结构（如存储过程或触发器）。
- 返回的是原始数据（例如元组），不与 Django 模型绑定。

特性	<code>raw()</code>	<code>connection.cursor()</code>
适用场景	模型绑定的 SELECT 查询	任意原生 SQL（包括写入、删除）
结果类型	Django 模型实例 (RawQuerySet)	原始查询结果（如元组）
复杂查询支持	仅支持与模型相关的简单查询	支持复杂查询和数据库特性
写入支持	不支持	完全支持（INSERT、UPDATE、DELETE 等）
易用性	简单，易于结合 ORM	灵活性高，但需要手动处理结果

SQL injection!

Using the `params` argument ([]) and leaving the placeholders (%s) unquoted protects you from SQL injection attacks

```
1 Person.objects.raw('SELECT * FROM myapp_person WHERE last_name = %s', [lname])
```

SQL injection: an example

query = "SELECT * FROM library_book WHERE borrower='%s' " % username

- What happens with input: `"; select true; --"`
`>>> select * from library_book where borrower=""; select true; --'`
- The resulting text contains three statements and it displays all the records of the users table.
 - `select * from library_book where borrower="";`
 - `select true;`
 - `--'` (This snippet defuses anything that comes after it. The intruder added the comment symbol (`--`) to turn everything you might have put after the last placeholder into a comment.)
- However, for `sqlite3`, the above query will result in the message: “you can only execute ONE statement”.
- <https://www.youtube.com/watch?v=TSqXkkOt6oM> – an example on login

10

——' (这段代码消除了它后面的任何内容。入侵者添加了注释符号 (——)，将您可能放在最后一个占位符之后的所有内容都转换为注释。

▼

SQL |

```
1 select * from library_book where borrower='';
2 select true;
3 -- (注释掉后续语句)
```

在 Django 中，推荐使用 `cursor.execute()` 的参数列表形式，它会自动转义非法字符，避免用户输入直接影响查询逻辑

▼

Python |

```
1 def search(request):
2     ...
3     cursor.execute("SELECT * FROM blog_post WHERE title LIKE %s", ['%' + query + '%'])
```

- 关键点：
 - `%s` 是 SQL 语句的占位符，表示需要动态填充的数据。
 - 第二个参数 `[query]` 是参数列表，`query` 的值会被安全地嵌入到查询中替换占位符。
- 安全注入过程：
 - 用户输入的数据会被数据库驱动程序转义，确保不会破坏 SQL 语法。
 - 即使用户输入恶意内容（如 `'; DELETE FROM blog_post;--`），查询只会将它视为字符串处理，而不会执行破坏性操作。

考试重点!

An example: RAW query vs ORM

```
def searchRAW(request):
    if 'q' in request.GET and request.GET['q']:
        query = request.GET['q']
        from django.db import connection
        with connection.cursor() as cur:
            cur.execute("SELECT * FROM blog_post WHERE title LIKE %s",
                        ['%' + query + '%'])
        posts = dictfetchall(cur) # this returns with their field names
        return render(request, 'search_results.html', {'posts': posts,
            'query': query})
    else:
        return HttpResponse('Please submit a search term.')
```

`path('post/search/', views.searchRAW, name='search'),`

```
<form action="/blog/post/search/" method="GET">
<input type = "text" name="q">
<input type = "submit" value = "search">
</form>
```

考试考这里! chapter10

```
def search(request):
    if 'q' in request.GET and request.GET['q']:
        query = request.GET['q']
        posts = Post.objects.filter(title__icontains=query)
        return render(request, 'search_results.html',
            {'posts': posts, 'query': query})
    else:
        return HttpResponse('Submit a search term.')
```

`path('post/search/', views.search, name='search'),`

Besides checking that 'q' exists in request.GET, also make sure that request.GET['q'] is a non-empty value before passing it to the database query.

对比RAW SQL查询和Django ORM查询的不同

(左) 主要逻辑:

- 通过 `request.GET` 获取查询参数 `q` , 判断是否存在有效值。
- 使用 `connection.cursor()` 直接执行原生 SQL 查询, 并使用 `%s` 占位符安全地传递参数。
- 查询结果通过自定义方法 `dictfetchall()` 转换为字典列表, 便于在模板中使用。

(右) Django ORM 查询

Python

```
1 def search(request):
2     if 'q' in request.GET and request.GET['q']:
3         query = request.GET['q']
4         posts = Post.objects.filter(title__icontains=query)
5         return render(request, 'search_results.html', {'posts': posts, 'query': query})
6     else:
7         return HttpResponse('Submit a search term.')
```

- 定义了一个名为 `search` 的视图函数, 接受 `request` 对象作为参数。
- `request.GET` 是一个字典样式的对象, 包含所有通过 GET 方法传递的参数。

`'q' in request.GET` 检查参数名 `q` 是否存在于用户请求中。

`request.GET['q']` 检查 `q` 的值是否非空。

如果用户提交了 `q` 参数且值有效，则继续执行，否则进入 `else` 块。

- 从请求中提取参数 `q` 的值，并将其赋值给变量 `query`；`query` 代表用户输入的搜索关键词。
- `Post.objects.filter()` 方法返回一个 `QuerySet`，对应数据库中满足查询条件的数据。
 - `title__icontains=query` 表示对 `Post` 模型的 `title` 字段进行模糊查询：
 - `icontains` 是一种字段查询表达式，表示大小写不敏感的子字符串匹配。
 - 如果 `query` 是 "Django"，它会匹配任何包含 "django" 或 "Django" 的标题。
- `posts` 是一个 `QuerySet`，包含符合条件的所有结果。

Django 的 `request.GET`

- `request.GET` 是一个 **类字典对象**，包含所有通过查询字符串传递的参数。
- 它是 Django 的 `HttpRequest` 对象的一部分，专门用于处理 GET 请求数据。

```
1 query = request.GET['q']
```

Returning results as a dictionary (dict)

- By default, the Python DB API returns results without their field names, which means you end up with a list of values, rather than a dict.
- A dictionary is a set of **key: value pairs**, separated with a comma. The keys are unique within one dictionary. A pair of braces creates an empty dictionary: {}.
- To return results as a dict by using something like this:

```
def dictfetchall(cursor):  
    columns = [col[0] for col in cursor.description]  
    return [  
        dict(zip(columns, row)) for row in cursor.fetchall()  
    ]
```

• Returns a tuple describing each column in the result row.
• The information about each column is a tuple of (column_name, declared_column_type), e.g. (('id', 'string'), ('parent_id', 'string'))

```
>>> list_1 = ['pie', 'cake', 'tart']  
>>> list_2 = ['apple', 'banana', 'lemon']  
>>> dict(zip(list_2, list_1))  
{'apple': 'pie', 'banana': 'cake', 'lemon': 'tart'}
```

the `zip` command turns two lists into one list, then the `dict` command returns it as a dictionary.

```
>>> cursor.execute("SELECT id, parent_id FROM test LIMIT 2");  
>>> cursor.fetchall()  
[(54360982, None), (54360880, None)]
```

```
>>> cursor.execute("SELECT id, parent_id FROM test LIMIT 2");  
>>> dictfetchall(cursor)  
[{'parent_id': None, 'id': 54360982}, {'parent_id': None, 'id': 54360880}]
```

默认情况下，Python 的 DB API 返回结果为列表，仅包含字段值。

`zip()` 和 `dict()`：

- `zip(columns, row)` 将字段名和数据值配对。
- `dict()` 将配对转换为字典。

Request vs Response

- HTTP messages are how data is exchanged between a server and a client.
- There are two types of messages:
 - *requests* sent by the client to trigger an action on the server, and
 - *responses*, the answer from the server.
- HTTP **requests** are messages **sent by the client** to initiate an action on the server. Their *start-line* contain three elements:
 - An *HTTP method*, a verb (like GET, PUT or POST) or a noun (like HEAD or OPTIONS), that describes the action to be performed.
 - The *request target*, usually a URL
 - The *HTTP version*, which defines the structure of the remaining message.

Post vs Get

- GET indicates that a resource should be fetched.
 - Use GET when the act of submitting the form is just a request to get data.
 - It results in a querystring starting with a ? to be appended at the URL together with the submitted data
- POST means that data is pushed to the server (creating or modifying a resource, or generating a temporary document to send back).
 - Use POST whenever the act of submitting the form will have some side effect—*changing* data, or sending an e-mail, or something else that's beyond simple *display* of data.